Project Report

Hydro-climatology Based Next-day Runoff Event Forecasting

Using NASA POWER Climate Data, SCS Curve Number Runoff Proxy, and LSTM Modelling

- Study area: Powai / IIT Bombay, Mumbai, India
- Coordinates used: 19.13° N, 72.91° E
- Data period: 2010-2025
- Forecast target: next-day runoff proxy event
- Model: Long Short-Term Memory sequence classifier
- Main performance result: ROC-AUC = 0.965, F1-score = 0.736

Generated on: 18 August 2026

Table of Contents

1. Abstract

Abstract

This project develops an end-to-end hydro-climatology workflow for forecasting next-day runoff-event risk near IIT Bombay/Powai, Mumbai. Daily climate variables from the NASA POWER API were used for the period 2010-2025. Since observed discharge measurements were not used, rainfall was transformed into a runoff-event proxy using the SCS Curve Number method. A Long Short-Term Memory model was then trained using 30-day sequences of hydro-climatic features to estimate the probability of a significant runoff event on the following day.

The independent test period was 2023-2025. The LSTM achieved an accuracy of 0.953, precision of 0.807, recall of 0.676, F1-score of 0.736, ROC-AUC of 0.965, and average precision of 0.753. Results indicate that recent rainfall, antecedent wetness, seasonal indicators, and weather variables provide useful information for classifying runoff-event risk in a monsoon-dominated urban environment.

2. Introduction and Problem Statement

Background

Mumbai experiences high rainfall concentration during the southwest monsoon. Short-duration and high-intensity rainfall can generate rapid surface runoff, urban drainage stress, waterlogging, and transport disruption. Anticipating runoff-event conditions even one day ahead can support risk awareness and planning.

Hydro-climatology combines climate variability, rainfall processes, catchment response, and water-related impacts. Machine learning methods are increasingly used in this domain because they can learn nonlinear relationships from historical sequences of rainfall and weather variables.

Problem statement

The central problem addressed here is whether recent hydro-climatic conditions can be used to estimate the probability of a significant runoff-event proxy on the next day for the Powai/IIT Bombay region.

3. Objectives and Scope

Objectives

- Acquire and process daily climate data for the Powai/IIT Bombay region.
- Generate hydrological features including antecedent rainfall and a runoff proxy.
- Define a next-day binary runoff-event target.
- Train an LSTM model using 30-day input sequences.
- Evaluate model performance using independent test-period data.
- Present visual results for hydro-climatic interpretation and model assessment.

Scope

The work is designed as a compact hydro-climatology modelling study. It does not attempt to replace detailed physically based hydrologic or hydraulic modelling. Instead, it demonstrates how data-driven sequence modelling can be combined with a simple hydrological runoff proxy to study next-day event risk.

4. Study Area and Data Description

Study area

The selected point represents Powai/IIT Bombay in Mumbai, Maharashtra, India. The approximate coordinates used for data extraction are 19.13° N latitude and 72.91° E longitude. The region is influenced by strong monsoon seasonality, with most annual rainfall occurring from June to September.

Data source

Daily climate data were obtained from the NASA POWER API. The selected variables include daily precipitation, mean temperature, maximum temperature, minimum temperature, relative humidity, and wind speed.

The analysis uses data from 1 January 2010 to 31 December 2025. Missing values represented by API fill values are replaced using forward and backward filling after chronological sorting.

Variables used

- PRECTOTCORR: corrected daily precipitation in mm/day.
- T2M, T2M_MAX, T2M_MIN: daily temperature indicators in °C.
- RH2M: relative humidity at 2 m in percent.
- WS2M: wind speed at 2 m in m/s.

5. Methodology

Rainfall-runoff proxy

A simplified SCS Curve Number method is used to estimate daily runoff depth from rainfall. The retention parameter is calculated as S = 25400 / CN - 254, where S is in mm and CN is the curve number. Initial abstraction is taken as Ia = 0.2S. When rainfall P exceeds Ia, runoff is computed as Q = (P - Ia)^2 / (P + 0.8S); otherwise Q = 0.

Antecedent 5-day rainfall is used to assign simplified dry, normal, and wet catchment conditions. This introduces a hydrological memory effect before the LSTM model is applied.

Target definition

A day is labelled as a runoff-event day when the calculated runoff proxy is at least 10 mm/day. The machine learning target is shifted by one day, so the model predicts whether tomorrow will be a runoff-event day.

Feature engineering

The input feature set includes rainfall, temperature variables, relative humidity, wind speed, runoff proxy, 5-day accumulated rainfall, and seasonal sine/cosine variables based on day-of-year. The seasonal features help represent the monsoon cycle numerically.

Sequence construction

Each model sample contains 30 consecutive days of input features. The corresponding output label describes whether the next day crosses the runoff-event threshold. This framing converts the problem into a supervised time-series classification task.

6. Model Development and Evaluation Strategy

LSTM model

Long Short-Term Memory networks are designed to learn patterns from sequences by maintaining a cell state and using gates to control information flow. The model used here contains input, forget, output, and candidate gates, followed by a sigmoid output layer for binary event probability.

The implementation uses NumPy to keep the underlying computations transparent. Weighted binary cross-entropy is used because runoff-event days are much fewer than non-event days. Adam optimization and gradient clipping are used during training.

Temporal data split

- Training set: 2010-2020
- Validation set: 2021-2022
- Test set: 2023-2025

A chronological split is used so that the model is evaluated on future years that were not used for training. This is important for realistic time-series assessment.

Evaluation metrics

Accuracy, precision, recall, F1-score, ROC-AUC, average precision, and confusion matrix are reported. The probability threshold is selected using the validation set by maximizing F1-score, and the final metrics are computed on the 2023-2025 test period.

Results and Performance Summary

The final model was evaluated on the independent 2023-2025 test period. The selected probability threshold was obtained from the validation period by maximizing F1-score. Since runoff-event days are relatively infrequent, precision, recall, F1-score, ROC-AUC and average precision are more informative than accuracy alone.

Metric	LSTM	Persistence baseline	Monsoon baseline
Accuracy	0.953	0.940	0.751
Precision	0.807	0.686	0.270
Recall	0.676	0.686	0.943
F1-score	0.736	0.686	0.420
ROC-AUC	0.965	—	—
Average precision	0.753	—	—
Selected threshold	0.925	—	—

Confusion Matrix for the LSTM Test Period

	Predicted no event	Predicted event
Actual no event	973	17
Actual event	34	71

Interpretation: the model correctly identifies many high-runoff-risk days while keeping the number of false alarms relatively low. The recall value shows that some event days are still missed, which is common in rare-event hydrological forecasting.

Figure 1. Monthly Rainfall, Runoff Proxy and Event Climatology

The monthly climatology shows that rainfall and runoff-event frequency are concentrated during the southwest monsoon months. This verifies that the processed data represent the expected hydro-climatic behaviour of Mumbai.

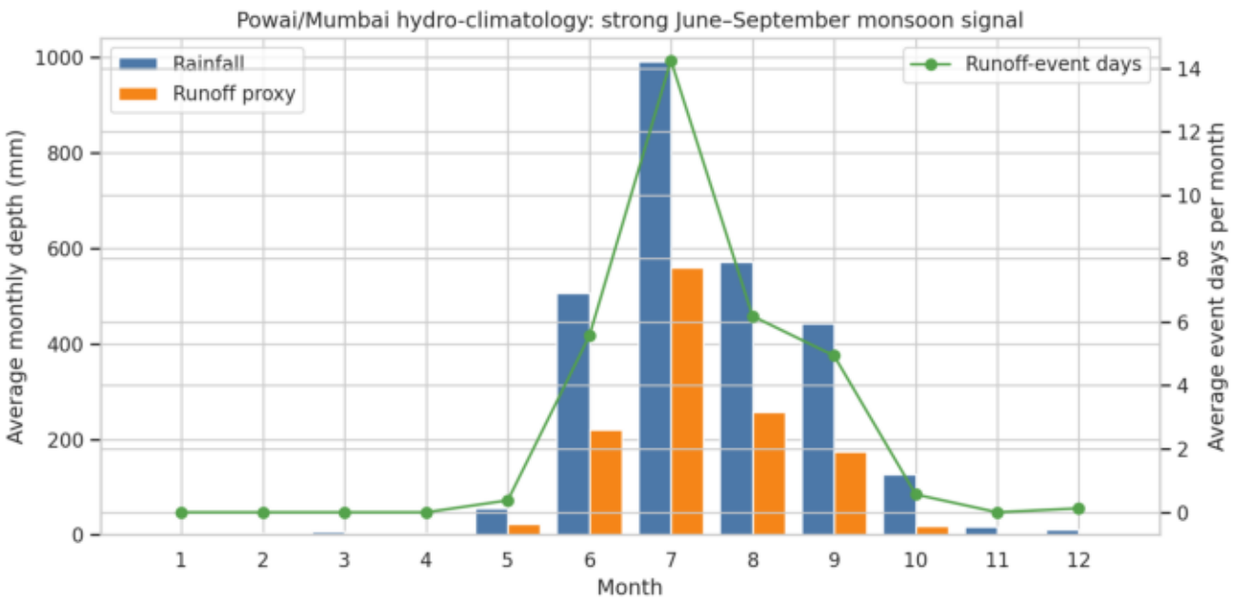


Figure 2. Annual Rainfall and Runoff-proxy Variability

Annual rainfall and runoff proxy values vary from year to year. This plot helps understand interannual variability in hydro-climatic forcing and event occurrence.

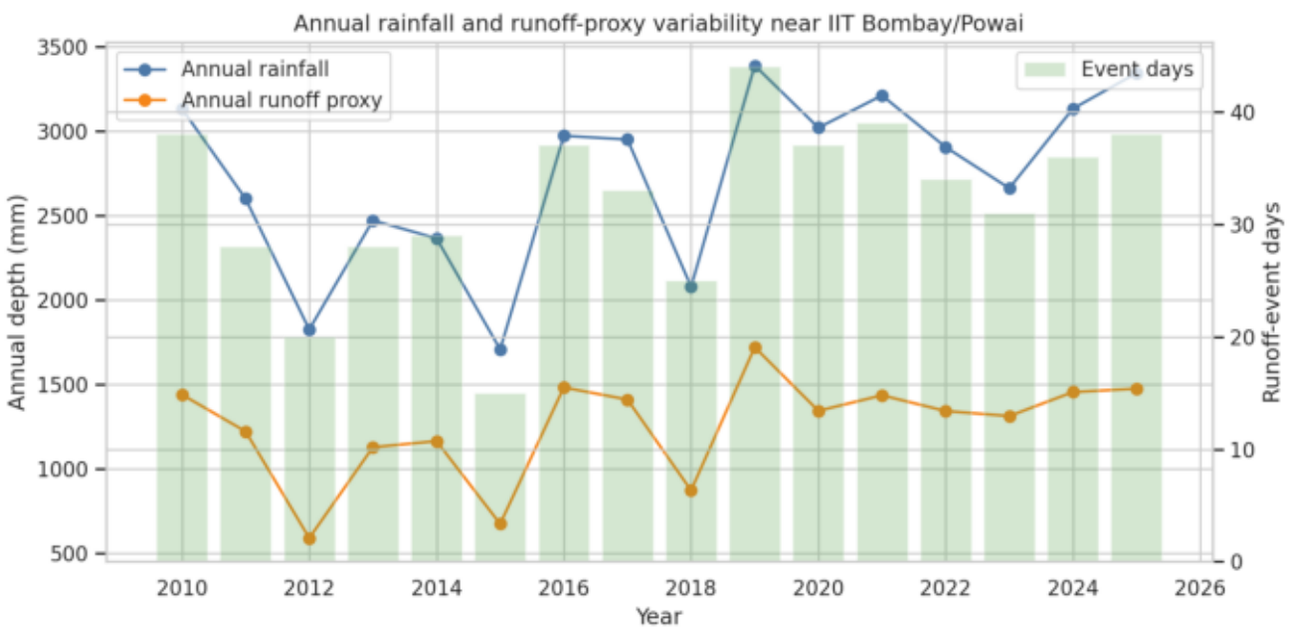


Figure 3. Daily Rainfall to Runoff Proxy during 2025 Monsoon

The 2025 monsoon zoom illustrates how high rainfall days are transformed into runoff-proxy peaks. The threshold line indicates the event definition used for classification.

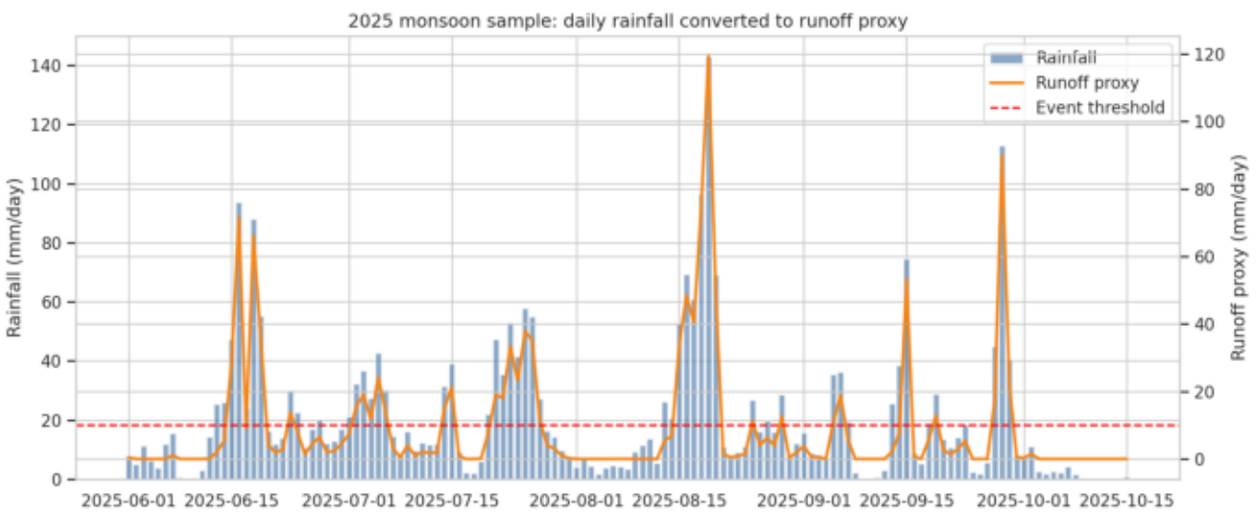


Figure 4. LSTM Training and Validation Loss

The learning curve presents weighted binary cross-entropy loss across epochs. It is used to check whether the model is learning from the training data and maintaining reasonable validation behaviour.

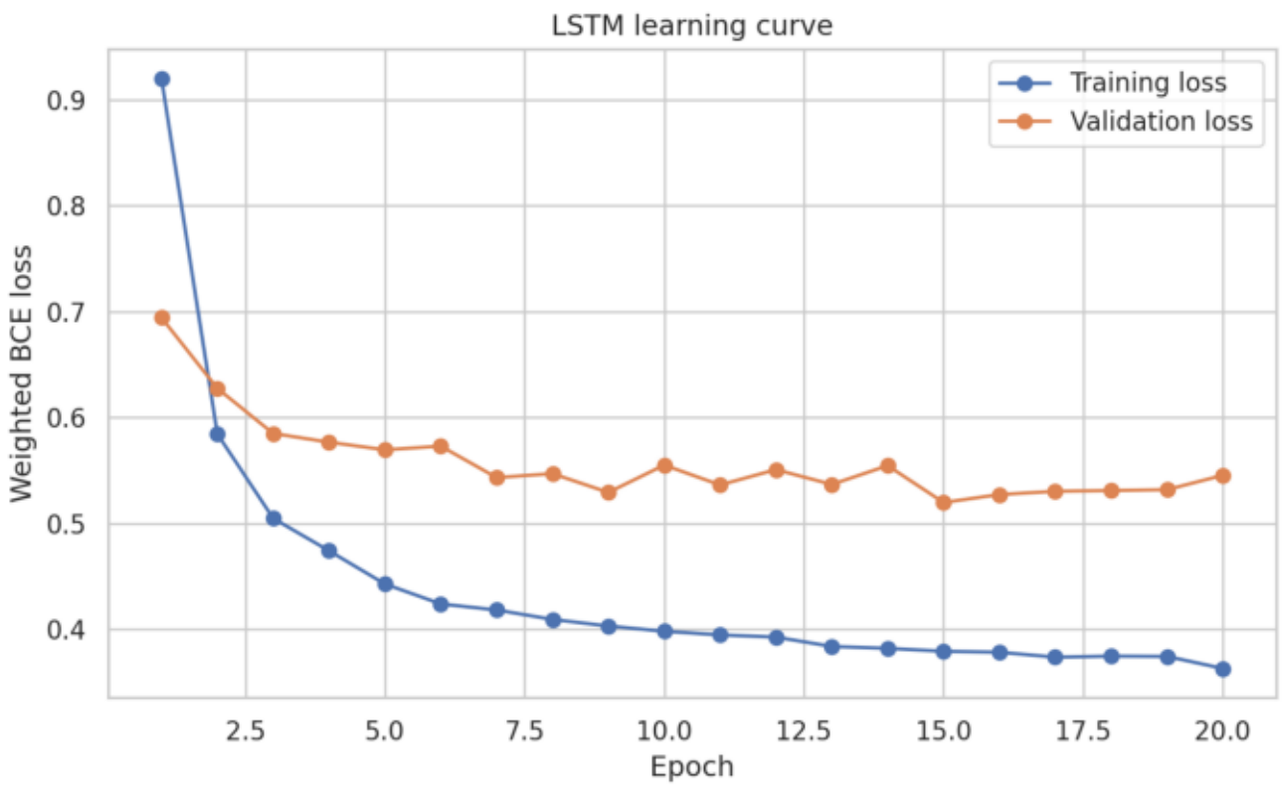


Figure 5. Predicted Event Probability during 2025 Monsoon

The probability curve is compared with actual event days. Increased probabilities around event periods indicate that the model is using antecedent rainfall and seasonal signals effectively.

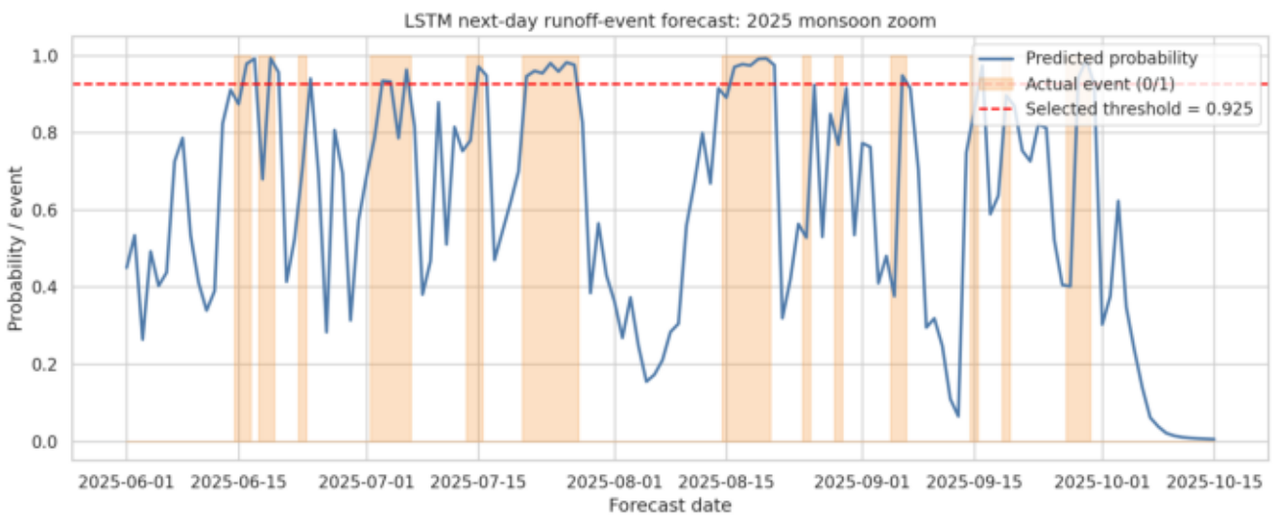


Figure 6. ROC and Precision-Recall Curves

The ROC and precision-recall curves summarize model performance over many probability thresholds. The ROC-AUC of 0.965 indicates strong ranking ability.

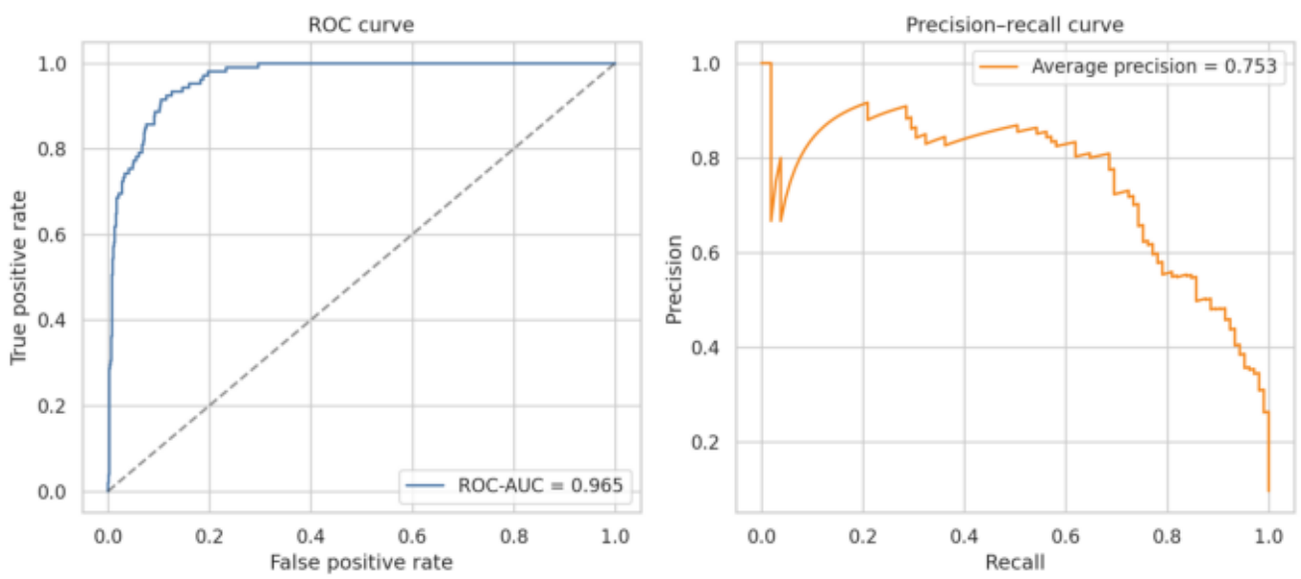


Figure 7. Confusion Matrix for Test Period

The confusion matrix shows correct and incorrect event classifications during 2023-2025. It provides a direct count-based interpretation of model decisions.

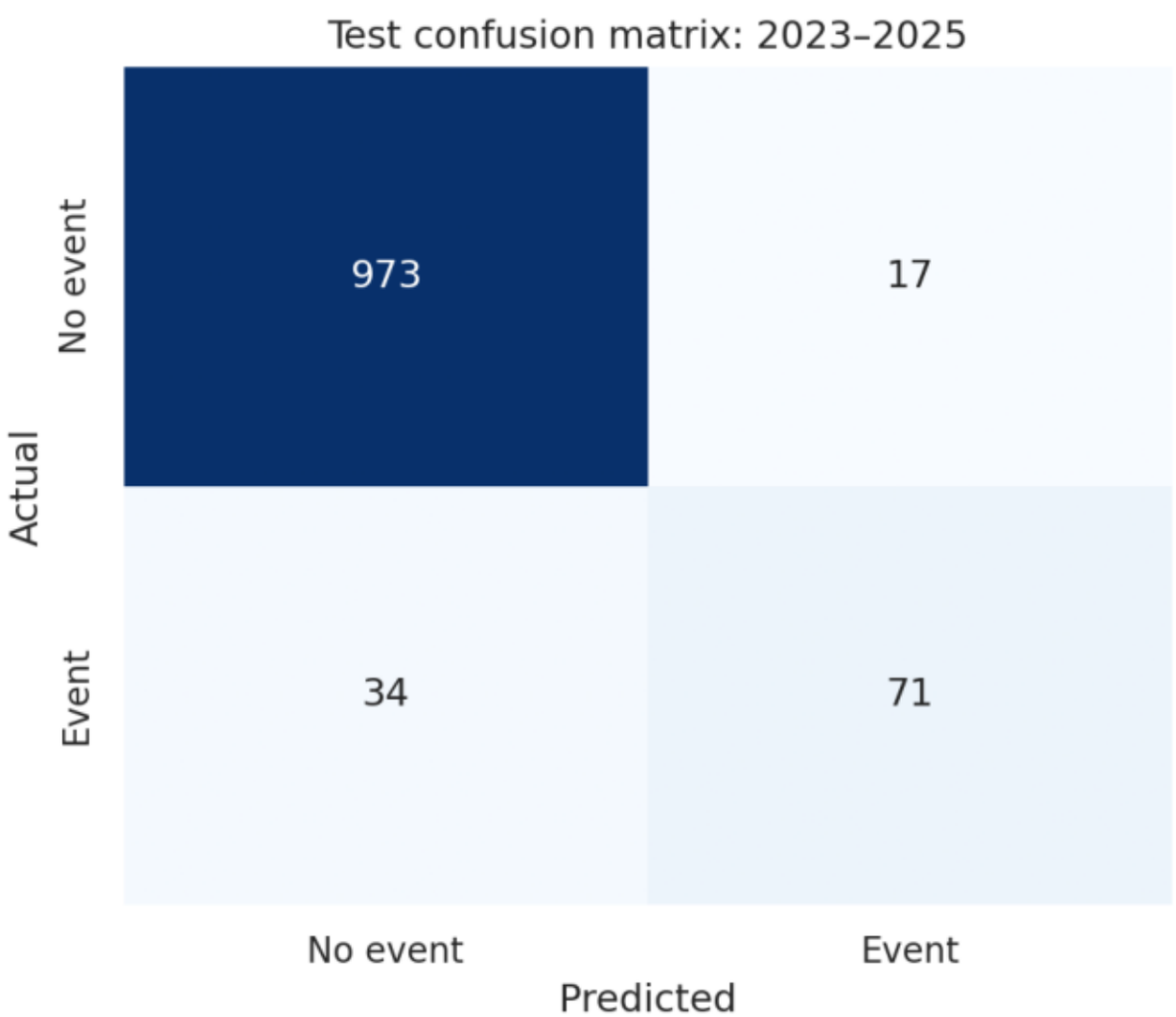
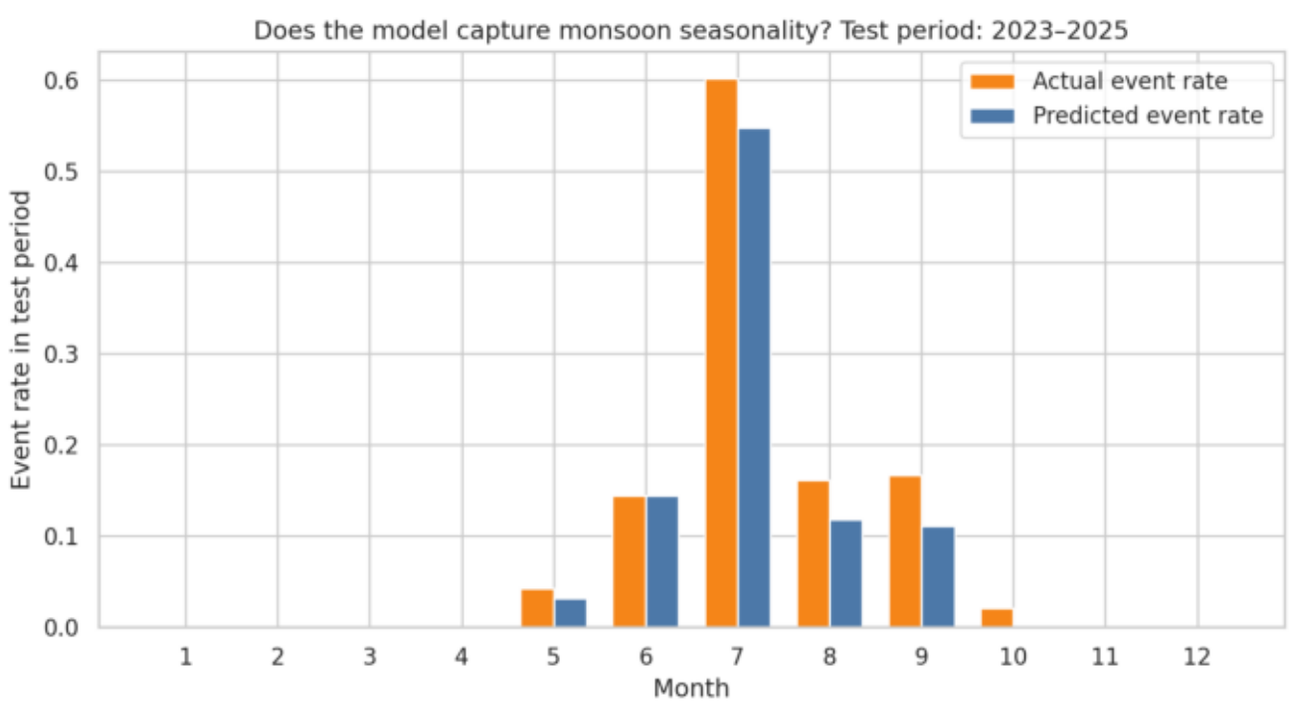


Figure 8. Monthly Actual and Predicted Event Rate

Monthly event-rate comparison assesses whether the model captures seasonal concentration of runoff-event risk, especially during the monsoon period.



8. Discussion

Hydro-climatic interpretation

The results confirm that the largest runoff-event probabilities occur during the monsoon period. This is physically reasonable because heavy rainfall and high antecedent wetness reduce catchment storage capacity and increase runoff response.

The inclusion of 5-day accumulated rainfall and a runoff proxy provides the model with information about antecedent moisture conditions. This is important because the same rainfall amount can produce different runoff responses depending on prior wetness.

Model interpretation

The LSTM performs better than simple baseline approaches because it uses sequential information from the previous 30 days rather than relying only on yesterday's event status or a fixed monsoon-month rule.

The test-period precision indicates that many model alarms are meaningful, while the recall indicates that some events remain difficult to detect. This trade-off is typical in rare-event forecasting problems.

9. Limitations and Future Scope

Limitations

- The target is a calculated runoff proxy rather than measured streamflow or drainage water level.
- The curve number values are simplified and do not represent a calibrated catchment model.
- NASA POWER data are gridded climate estimates and may differ from local rain-gauge observations.
- The model is trained for a single point location and does not include spatial variability across Mumbai.

Future scope

- Use observed rainfall from local stations or IMD data for improved local accuracy.
- Include measured streamflow, lake level, or urban drainage water-level data if available.
- Add land use, soil type, slope, imperviousness, and drainage network features.
- Compare LSTM with GRU, temporal convolution, Random Forest, Logistic Regression, and gradient boosting models.
- Extend the task from binary classification to runoff-depth regression or multi-class event severity prediction.

10. Conclusion

Conclusion

This project demonstrates a complete hydro-climatology modelling workflow for next-day runoff-event forecasting near IIT Bombay/Powai. Public climate data were processed, hydrological features were engineered, a runoff-event proxy was created, and an LSTM sequence model was trained and evaluated using a chronological split.

The final test performance, including ROC-AUC of 0.965 and F1-score of 0.736, indicates that the model captures meaningful hydro-climatic patterns associated with monsoon runoff-event risk. The study also highlights the importance of clearly distinguishing proxy-based event prediction from observed discharge forecasting.

11. References

References

1. NASA POWER Project. Prediction Of Worldwide Energy Resources daily climate data API documentation.

2. USDA Soil Conservation Service. National Engineering Handbook, Section 4: Hydrology. Curve Number rainfall-runoff method.

3. Hochreiter, S. and Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735-1780.

4. Wilks, D. S. Statistical Methods in the Atmospheric Sciences. Academic Press.

5. Chow, V. T., Maidment, D. R., and Mays, L. W. Applied Hydrology. McGraw-Hill.

Appendix A: Complete Python Source Code

File: src/hydro_lstm_project.py

```
0001: """
0002: Hydro-climatology self project for beginners
0003: ------------------------------------------
0004: Project: Forecast next-day runoff-event risk in Powai/Mumbai using NASA POWER daily
0005: climate data and a small Long Short-Term Memory (LSTM) neural network implemented
0006: only with NumPy.
0007:
0008: Run from the project root:
0009:     python src/hydro_lstm_project.py
0010:
0011: The script downloads/loads data, creates a simple SCS-Curve-Number runoff proxy,
0012: trains the LSTM, evaluates it on a time-based test period, and saves plots in
0013: results/.
0014: """
0015:
0016: from __future__ import annotations
0017:
0018: import json
0019: import math
0020: import warnings
0021: from dataclasses import dataclass
0022: from pathlib import Path
0023:
0024: import matplotlib.pyplot as plt
0025: import numpy as np
0026: import pandas as pd
0027: import requests
0028: import seaborn as sns
0029: from sklearn.metrics import (
0030:     accuracy_score,
0031:     average_precision_score,
0032:     confusion_matrix,
0033:     precision_recall_curve,
0034:     precision_recall_fscore_support,
0035:     roc_auc_score,
0036:     roc_curve,
0037: )
0038: from sklearn.preprocessing import StandardScaler
0039:
0040: warnings.filterwarnings("ignore")
0041: sns.set_theme(style="whitegrid", context="notebook")
0042:
0043: # ------------------------------
0044: # 1. Project configuration
0045: # ------------------------------
0046:
0047: ROOT = Path(__file__).resolve().parents[1]
0048: DATA_DIR = ROOT / "data"
0049: RESULTS_DIR = ROOT / "results"
0050: DATA_DIR.mkdir(exist_ok=True)
0051: RESULTS_DIR.mkdir(exist_ok=True)
0052:
0053: # Powai/IIT Bombay approximate coordinates
0054: LATITUDE = 19.13
0055: LONGITUDE = 72.91
0056: START_DATE = "20100101"
0057: END_DATE = "20251231"
0058: LOOKBACK_DAYS = 30
0059: RUNOFF_EVENT_THRESHOLD_MM = 10.0
0060:
0061: NASA_POWER_PARAMETERS = [
0062:     "PRECTOTCORR",  # precipitation corrected, mm/day
```

Appendix A: Complete Python Source Code

File: src/hydro_lstm_project.py

```
0063:     "T2M",          # temperature at 2 m, deg C
0064:     "T2M_MAX",      # max temperature, deg C
0065:     "T2M_MIN",      # min temperature, deg C
0066:     "RH2M",         # relative humidity at 2 m, %
0067:     "WS2M",         # wind speed at 2 m, m/s
0068: ]
0069:
0070: FEATURE_COLUMNS = [
0071:     "PRECTOTCORR",
0072:     "T2M",
0073:     "T2M_MAX",
0074:     "T2M_MIN",
0075:     "RH2M",
0076:     "WS2M",
0077:     "runoff_mm",
0078:     "rain_5day_mm",
0079:     "doy_sin",
0080:     "doy_cos",
0081: ]
0082:
0083:
0084: # ------------------------------
0085: # 2. A tiny LSTM implemented in NumPy
0086: # ------------------------------
0087:
0088: class NumpyLSTMClassifier:
0089:     """A compact educational LSTM for binary sequence classification.
0090:
0091:     It uses the standard LSTM gates:
0092:         input gate, forget gate, output gate, and candidate cell state.
0093:
0094:     This is intentionally written without TensorFlow/PyTorch so that beginners can
0095:     inspect the equations directly. For a larger project, PyTorch/Keras would be
0096:     more convenient.
0097:     """
0098:
0099:     def __init__(
0100:         self,
0101:         input_size: int,
0102:         hidden_size: int = 16,
0103:         learning_rate: float = 0.003,
0104:         seed: int = 42,
0105:         grad_clip: float = 1.0,
0106:     ) -> None:
0107:         rng = np.random.default_rng(seed)
0108:         self.input_size = input_size
0109:         self.hidden_size = hidden_size
0110:         self.learning_rate = learning_rate
0111:         self.grad_clip = grad_clip
0112:         self.step = 0
0113:
0114:         # Combined weights for four gates: input, forget, output, candidate.
0115:         self.params = {
0116:             "Wx": rng.normal(0, 0.5 / np.sqrt(input_size), (input_size, 4 * hidden_size)),
0117:             "Wh": rng.normal(0, 0.5 / np.sqrt(hidden_size), (hidden_size, 4 * hidden_size)),
0118:             "b": np.zeros((4 * hidden_size,)),
0119:             "Wy": rng.normal(0, 0.5 / np.sqrt(hidden_size), (hidden_size, 1)),
0120:             "by": np.zeros((1,)),
0121:         }
0122:
0123:         # Adam optimizer state
0124:         self.m = {k: np.zeros_like(v) for k, v in self.params.items()}
```

Appendix A: Complete Python Source Code

File: src/hydro_lstm_project.py

```
        self.v = {k: np.zeros_like(v) for k, v in self.params.items()}

    @staticmethod
    def sigmoid(x: np.ndarray) -> np.ndarray:
        x = np.clip(x, -40, 40)
        return 1.0 / (1.0 + np.exp(-x))

    def forward(self, X: np.ndarray):
        """Forward pass.

        X shape: (batch, time_steps, features)
        returns: logits and a list of cached values for backpropagation
        """
        n, time_steps, _ = X.shape
        h = np.zeros((n, self.hidden_size))
        c = np.zeros((n, self.hidden_size))
        caches = []

        for t in range(time_steps):
            x_t = X[:, t, :]
            h_prev, c_prev = h, c

            z = x_t @ self.params["Wx"] + h_prev @ self.params["Wh"] + self.params["b"]
            H = self.hidden_size
            i = self.sigmoid(z[:, :H])
            f = self.sigmoid(z[:, H : 2 * H])
            o = self.sigmoid(z[:, 2 * H : 3 * H])
            g = np.tanh(z[:, 3 * H :])

            c = f * c_prev + i * g
            h = o * np.tanh(c)
            caches.append((x_t, h_prev, c_prev, i, f, o, g, c, h))

        logits = h @ self.params["Wy"] + self.params["by"]
        return logits, caches

    def loss_and_gradients(self, X: np.ndarray, y: np.ndarray, pos_weight: float):
        """Weighted binary cross entropy and manual backpropagation through time."""
        y = y.reshape(-1, 1).astype(float)
        n = len(y)
        H = self.hidden_size

        logits, caches = self.forward(X)
        prob = self.sigmoid(logits)
        eps = 1e-8
        weights = np.where(y == 1, pos_weight, 1.0)
        loss = -np.mean(weights * (y * np.log(prob + eps) + (1 - y) * np.log(1 - prob + eps)))

        # dL/dlogits for weighted BCE
        dlogits = weights * (prob - y) / n

        grads = {k: np.zeros_like(v) for k, v in self.params.items()}
        h_last = caches[-1][-1]
        grads["Wy"] = h_last.T @ dlogits
        grads["by"] = dlogits.sum(axis=0)

        dh = dlogits @ self.params["Wy"].T
        dh_next = np.zeros_like(dh)
        dc_next = np.zeros_like(dh)

        for cache in reversed(caches):
            x_t, h_prev, c_prev, i, f, o, g, c, h = cache
```

Appendix A: Complete Python Source Code

File: src/hydro_lstm_project.py

```
            dh_total = dh + dh_next
            tanh_c = np.tanh(c)

            do = dh_total * tanh_c
            dc = dh_total * o * (1 - tanh_c**2) + dc_next

            df = dc * c_prev
            dc_next = dc * f
            di = dc * g
            dg = dc * i

            dzi = di * i * (1 - i)
            dzf = df * f * (1 - f)
            dzo = do * o * (1 - o)
            dzg = dg * (1 - g**2)
            dz = np.concatenate([dzi, dzf, dzo, dzg], axis=1)

            grads["Wx"] += x_t.T @ dz
            grads["Wh"] += h_prev.T @ dz
            grads["b"] += dz.sum(axis=0)

            dh_next = dz @ self.params["Wh"].T
            dh = np.zeros_like(dh)

        # Gradient clipping prevents exploding gradients in sequence models.
        for key in grads:
            grads[key] = np.clip(grads[key], -self.grad_clip, self.grad_clip)

        return float(loss), grads

    def adam_update(self, grads: dict[str, np.ndarray]) -> None:
        beta1, beta2, eps = 0.9, 0.999, 1e-8
        self.step += 1
        for key, grad in grads.items():
            self.m[key] = beta1 * self.m[key] + (1 - beta1) * grad
            self.v[key] = beta2 * self.v[key] + (1 - beta2) * grad * grad
            m_hat = self.m[key] / (1 - beta1**self.step)
            v_hat = self.v[key] / (1 - beta2**self.step)
            self.params[key] -= self.learning_rate * m_hat / (np.sqrt(v_hat) + eps)

    def fit(
        self,
        X_train: np.ndarray,
        y_train: np.ndarray,
        X_val: np.ndarray,
        y_val: np.ndarray,
        epochs: int = 20,
        batch_size: int = 64,
        seed: int = 0,
    ) -> pd.DataFrame:
        pos_weight = (len(y_train) - y_train.sum()) / (y_train.sum() + 1e-8)
        rng = np.random.default_rng(seed)
        history = []

        for epoch in range(1, epochs + 1):
            order = rng.permutation(len(X_train))
            batch_losses = []

            for start in range(0, len(order), batch_size):
                batch_idx = order[start : start + batch_size]
                loss, grads = self.loss_and_gradients(X_train[batch_idx], y_train[batch_idx], pos_weight)
                self.adam_update(grads)
```

Appendix A: Complete Python Source Code

File: src/hydro_lstm_project.py

```
0249:                 batch_losses.append(loss)
0250:
0251:             train_loss = float(np.mean(batch_losses))
0252:             val_loss = weighted_bce(y_val, self.predict_proba(X_val), pos_weight)
0253:             history.append({"epoch": epoch, "train_loss": train_loss, "val_loss": val_loss})
0254:             print(f"Epoch {epoch:02d}/{epochs} | train loss={train_loss:.4f} | val loss={val_loss:.4f}")
0255:
0256:         return pd.DataFrame(history)
0257:
0258:     def predict_proba(self, X: np.ndarray) -> np.ndarray:
0259:         logits, _ = self.forward(X)
0260:         return self.sigmoid(logits).ravel()
0261:
0262:
0263: def weighted_bce(y_true: np.ndarray, prob: np.ndarray, pos_weight: float) -> float:
0264:     y = y_true.reshape(-1, 1).astype(float)
0265:     p = prob.reshape(-1, 1)
0266:     eps = 1e-8
0267:     weights = np.where(y == 1, pos_weight, 1.0)
0268:     return float(-np.mean(weights * (y * np.log(p + eps) + (1 - y) * np.log(1 - p + eps))))
0269:
0270:
0271: # ------------------------------
0272: # 3. Data download and hydrology features
0273: # ------------------------------
0274:
0275: def download_nasa_power_data() -> pd.DataFrame:
0276:     """Download daily climate data from NASA POWER or load cached CSV."""
0277:     csv_path = DATA_DIR / "nasa_power_powai_2010_2025.csv"
0278:     if csv_path.exists():
0279:         print(f"Loading cached data: {csv_path}")
0280:         df = pd.read_csv(csv_path, parse_dates=["date"]).set_index("date")
0281:         return df
0282:
0283:     print("Downloading NASA POWER daily data...")
0284:     url = "https://power.larc.nasa.gov/api/temporal/daily/point"
0285:     params = {
0286:         "parameters": ",".join(NASA_POWER_PARAMETERS),
0287:         "community": "AG",
0288:         "longitude": LONGITUDE,
0289:         "latitude": LATITUDE,
0290:         "start": START_DATE,
0291:         "end": END_DATE,
0292:         "format": "JSON",
0293:     }
0294:     response = requests.get(url, params=params, timeout=60)
0295:     response.raise_for_status()
0296:     raw = response.json()["properties"]["parameter"]
0297:
0298:     df = pd.DataFrame({key: pd.Series(value) for key, value in raw.items()})
0299:     df.index = pd.to_datetime(df.index, format="%Y%m%d")
0300:     df.index.name = "date"
0301:     df = df.sort_index().replace(-999, np.nan).ffill().bfill()
0302:     df.to_csv(csv_path)
0303:     print(f"Saved data: {csv_path}")
0304:     return df
0305:
0306:
0307: def add_hydrology_features(df: pd.DataFrame) -> pd.DataFrame:
0308:     """Create runoff proxy and time-series features.
0309:
0310:     We use the SCS Curve Number idea as a simple hydrological approximation:
```

Appendix A: Complete Python Source Code

File: src/hydro_lstm_project.py

```
    rainfall first satisfies initial abstraction; remaining water becomes runoff.

    Because this beginner-level study does not use local observed discharge data, the
    target is named a 'runoff-event proxy', not measured river discharge.
    """
    df = df.copy()
    rainfall = df["PRECTOTCORR"].to_numpy(dtype=float)

    # Antecedent rainfall decides whether the catchment is dry/normal/wet.
    previous_5day = (
        pd.Series(rainfall, index=df.index)
        .rolling(5, min_periods=1)
        .sum()
        .shift(1)
        .fillna(0)
        .to_numpy()
    )

    # Educational AMC classes for an urban/suburban catchment.
    curve_number = np.where(previous_5day < 35, 80, np.where(previous_5day > 53, 92, 86))
    storage_s = 25400 / curve_number - 254  # mm
    initial_abstraction = 0.2 * storage_s
    runoff = np.where(
        rainfall > initial_abstraction,
        (rainfall - initial_abstraction) ** 2 / (rainfall + 0.8 * storage_s),
        0.0,
    )

    df["curve_number"] = curve_number
    df["runoff_mm"] = runoff
    df["rain_5day_mm"] = pd.Series(rainfall, index=df.index).rolling(5, min_periods=1).sum()
    df["runoff_event"] = (df["runoff_mm"] >= RUNOFF_EVENT_THRESHOLD_MM).astype(int)

    # Seasonal cycle as numeric variables. This helps the model learn monsoon seasonality.
    day_of_year = df.index.dayofyear.to_numpy()
    df["doy_sin"] = np.sin(2 * np.pi * day_of_year / 366)
    df["doy_cos"] = np.cos(2 * np.pi * day_of_year / 366)

    # Forecast target: whether tomorrow's runoff proxy crosses the selected threshold.
    df["target_runoff_event_tomorrow"] = df["runoff_event"].shift(-1)
    return df


@dataclass
class DatasetBundle:
    df: pd.DataFrame
    X_train: np.ndarray
    y_train: np.ndarray
    dates_train: pd.DatetimeIndex
    X_val: np.ndarray
    y_val: np.ndarray
    dates_val: pd.DatetimeIndex
    X_test: np.ndarray
    y_test: np.ndarray
    dates_test: pd.DatetimeIndex
    persistence_test: np.ndarray
    seasonal_test: np.ndarray


def make_lstm_dataset(df: pd.DataFrame) -> DatasetBundle:
    """Create 30-day sequences and time-based train/validation/test splits."""
    df = df.dropna(subset=["target_runoff_event_tomorrow"]).copy()
```

Appendix A: Complete Python Source Code

File: src/hydro_lstm_project.py

```
0373:     feature_array = df[FEATURE_COLUMNS].to_numpy(dtype=float)
0374:     target_array = df["target_runoff_event_tomorrow"].to_numpy(dtype=int)
0375:     current_runoff_event = df["runoff_event"].to_numpy(dtype=int)
0376:     all_dates = df.index
0377:
0378:     X, y, forecast_dates, persistence = [], [], [], []
0379:     for end_idx in range(LOOKBACK_DAYS - 1, len(df) - 1):
0380:         X.append(feature_array[end_idx - LOOKBACK_DAYS + 1 : end_idx + 1])
0381:         y.append(target_array[end_idx])
0382:         # The forecast date is tomorrow, because y is tomorrow's event.
0383:         forecast_dates.append(all_dates[end_idx + 1])
0384:         persistence.append(current_runoff_event[end_idx])
0385:
0386:     X = np.array(X, dtype=float)
0387:     y = np.array(y, dtype=int)
0388:     forecast_dates = pd.DatetimeIndex(forecast_dates)
0389:     persistence = np.array(persistence, dtype=int)
0390:
0391:     train_mask = forecast_dates.year <= 2020
0392:     val_mask = (forecast_dates.year >= 2021) & (forecast_dates.year <= 2022)
0393:     test_mask = forecast_dates.year >= 2023
0394:
0395:     scaler = StandardScaler()
0396:     scaler.fit(X[train_mask].reshape(-1, X.shape[2]))
0397:     X_scaled = scaler.transform(X.reshape(-1, X.shape[2])).reshape(X.shape)
0398:
0399:     # Simple baseline: predict event on all monsoon dates.
0400:     seasonal = np.isin(forecast_dates.month, [6, 7, 8, 9]).astype(int)
0401:
0402:     return DatasetBundle(
0403:         df=df,
0404:         X_train=X_scaled[train_mask],
0405:         y_train=y[train_mask],
0406:         dates_train=forecast_dates[train_mask],
0407:         X_val=X_scaled[val_mask],
0408:         y_val=y[val_mask],
0409:         dates_val=forecast_dates[val_mask],
0410:         X_test=X_scaled[test_mask],
0411:         y_test=y[test_mask],
0412:         dates_test=forecast_dates[test_mask],
0413:         persistence_test=persistence[test_mask],
0414:         seasonal_test=seasonal[test_mask],
0415:     )
0416:
0417:
0418: # ------------------------------
0419: # 4. Evaluation utilities and plots
0420: # ------------------------------
0421:
0422: def choose_threshold(y_val: np.ndarray, prob_val: np.ndarray) -> tuple[float, dict]:
0423:     """Choose a probability threshold that maximizes validation F1-score."""
0424:     thresholds = np.linspace(0.05, 0.95, 181)
0425:     best = {"threshold": 0.5, "precision": 0.0, "recall": 0.0, "f1": -1.0}
0426:     for threshold in thresholds:
0427:         pred = (prob_val >= threshold).astype(int)
0428:         precision, recall, f1, _ = precision_recall_fscore_support(
0429:             y_val, pred, average="binary", zero_division=0
0430:         )
0431:         if f1 > best["f1"]:
0432:             best = {
0433:                 "threshold": float(threshold),
0434:                 "precision": float(precision),
```

Appendix A: Complete Python Source Code

File: src/hydro_lstm_project.py

```
0435:                 "recall": float(recall),
0436:                 "f1": float(f1),
0437:             }
0438:     return best["threshold"], best
0439:
0440:
0441: def binary_metrics(y_true: np.ndarray, pred: np.ndarray, prob: np.ndarray | None = None) -> dict:
0442:     precision, recall, f1, _ = precision_recall_fscore_support(
0443:         y_true, pred, average="binary", zero_division=0
0444:     )
0445:     out = {
0446:         "accuracy": float(accuracy_score(y_true, pred)),
0447:         "precision": float(precision),
0448:         "recall": float(recall),
0449:         "f1": float(f1),
0450:         "confusion_matrix": confusion_matrix(y_true, pred).tolist(),
0451:         "event_rate": float(np.mean(y_true)),
0452:     }
0453:     if prob is not None and len(np.unique(y_true)) > 1:
0454:         out["roc_auc"] = float(roc_auc_score(y_true, prob))
0455:         out["average_precision"] = float(average_precision_score(y_true, prob))
0456:     return out
0457:
0458:
0459: def save_metrics(metrics: dict) -> None:
0460:     with open(RESULTS_DIR / "metrics.json", "w", encoding="utf-8") as f:
0461:         json.dump(metrics, f, indent=2)
0462:
0463:
0464: def plot_monthly_climatology(df: pd.DataFrame) -> None:
0465:     monthly = df.groupby(df.index.month).agg(
0466:         mean_rain_mm=("PRECTOTCORR", "sum"),
0467:         mean_runoff_mm=("runoff_mm", "sum"),
0468:         event_days=("runoff_event", "sum"),
0469:     )
0470:     years = df.index.year.nunique()
0471:     monthly["mean_rain_mm"] /= years
0472:     monthly["mean_runoff_mm"] /= years
0473:     monthly["event_days"] /= years
0474:
0475:     fig, ax1 = plt.subplots(figsize=(10, 5))
0476:     months = np.arange(1, 13)
0477:     ax1.bar(months - 0.2, monthly["mean_rain_mm"], width=0.4, label="Rainfall", color="#4C78A8")
0478:     ax1.bar(months + 0.2, monthly["mean_runoff_mm"], width=0.4, label="Runoff proxy", color="#F58518")
0479:     ax1.set_ylabel("Average monthly depth (mm)")
0480:     ax1.set_xlabel("Month")
0481:     ax1.set_xticks(months)
0482:     ax1.set_title("Powai/Mumbai hydro-climatology: strong June–September monsoon signal")
0483:     ax1.legend(loc="upper left")
0484:
0485:     ax2 = ax1.twinx()
0486:     ax2.plot(months, monthly["event_days"], color="#54A24B", marker="o", label="Runoff-event days")
0487:     ax2.set_ylabel("Average event days per month")
0488:     ax2.legend(loc="upper right")
0489:     fig.tight_layout()
0490:     fig.savefig(RESULTS_DIR / "01_monthly_rainfall_runoff_climatology.png", dpi=180)
0491:     plt.close(fig)
0492:
0493:
0494: def plot_annual_trend(df: pd.DataFrame) -> None:
0495:     annual = df.groupby(df.index.year).agg(
0496:         rain_mm=("PRECTOTCORR", "sum"), runoff_mm=("runoff_mm", "sum"), event_days=("runoff_event", "sum")
```

Appendix A: Complete Python Source Code

File: src/hydro_lstm_project.py

```
0497:     )
0498:     fig, ax1 = plt.subplots(figsize=(10, 5))
0499:     ax1.plot(annual.index, annual["rain_mm"], marker="o", color="#4C78A8", label="Annual rainfall")
0500:     ax1.plot(annual.index, annual["runoff_mm"], marker="o", color="#F58518", label="Annual runoff proxy")
0501:     ax1.set_ylabel("Annual depth (mm)")
0502:     ax1.set_xlabel("Year")
0503:     ax1.set_title("Annual rainfall and runoff-proxy variability near IIT Bombay/Powai")
0504:     ax1.legend(loc="upper left")
0505:     ax2 = ax1.twinx()
0506:     ax2.bar(annual.index, annual["event_days"], color="#54A24B", alpha=0.25, label="Event days")
0507:     ax2.set_ylabel("Runoff-event days")
0508:     ax2.legend(loc="upper right")
0509:     fig.tight_layout()
0510:     fig.savefig(RESULTS_DIR / "02_annual_rainfall_runoff_variability.png", dpi=180)
0511:     plt.close(fig)
0512:
0513:
0514: def plot_monsoon_sample(df: pd.DataFrame) -> None:
0515:     sample = df.loc["2025-06-01":"2025-10-15"]
0516:     fig, ax1 = plt.subplots(figsize=(12, 5))
0517:     ax1.bar(sample.index, sample["PRECTOTCORR"], color="#4C78A8", alpha=0.65, label="Rainfall")
0518:     ax1.set_ylabel("Rainfall (mm/day)")
0519:     ax1.set_title("2025 monsoon sample: daily rainfall converted to runoff proxy")
0520:     ax2 = ax1.twinx()
0521:     ax2.plot(sample.index, sample["runoff_mm"], color="#F58518", linewidth=2, label="Runoff proxy")
0522:     ax2.axhline(RUNOFF_EVENT_THRESHOLD_MM, color="red", linestyle="--", linewidth=1.4, label="Event
      threshold")
0523:     ax2.set_ylabel("Runoff proxy (mm/day)")
0524:     lines, labels = ax1.get_legend_handles_labels()
0525:     lines2, labels2 = ax2.get_legend_handles_labels()
0526:     ax1.legend(lines + lines2, labels + labels2, loc="upper right")
0527:     fig.tight_layout()
0528:     fig.savefig(RESULTS_DIR / "03_2025_monsoon_rainfall_to_runoff_proxy.png", dpi=180)
0529:     plt.close(fig)
0530:
0531:
0532: def plot_training_loss(history: pd.DataFrame) -> None:
0533:     fig, ax = plt.subplots(figsize=(8, 5))
0534:     ax.plot(history["epoch"], history["train_loss"], marker="o", label="Training loss")
0535:     ax.plot(history["epoch"], history["val_loss"], marker="o", label="Validation loss")
0536:     ax.set_xlabel("Epoch")
0537:     ax.set_ylabel("Weighted BCE loss")
0538:     ax.set_title("LSTM learning curve")
0539:     ax.legend()
0540:     fig.tight_layout()
0541:     fig.savefig(RESULTS_DIR / "04_lstm_training_loss.png", dpi=180)
0542:     plt.close(fig)
0543:
0544:
0545: def plot_test_probability(predictions: pd.DataFrame, threshold: float) -> None:
0546:     # Zoom to a monsoon period, where most hydrological decisions matter.
0547:     zoom = predictions.loc["2025-06-01":"2025-10-15"]
0548:     fig, ax = plt.subplots(figsize=(12, 5))
0549:     ax.plot(zoom.index, zoom["lstm_probability"], color="#4C78A8", linewidth=2, label="Predicted probability")
0550:     ax.fill_between(zoom.index, 0, zoom["actual_event"], step="mid", alpha=0.25, color="#F58518",
      label="Actual event (0/1)")
0551:     ax.axhline(threshold, color="red", linestyle="--", label=f"Selected threshold = {threshold:.3f}")
0552:     ax.set_ylim(-0.05, 1.05)
0553:     ax.set_ylabel("Probability / event")
0554:     ax.set_xlabel("Forecast date")
0555:     ax.set_title("LSTM next-day runoff-event forecast: 2025 monsoon zoom")
0556:     ax.legend(loc="upper right")
```

Appendix A: Complete Python Source Code

File: src/hydro_lstm_project.py

```
0557:     fig.tight_layout()
0558:     fig.savefig(RESULTS_DIR / "05_lstm_test_probabilities_2025_monsoon_zoom.png", dpi=180)
0559:     plt.close(fig)
0560:
0561:
0562: def plot_roc_pr(y_test: np.ndarray, prob_test: np.ndarray) -> None:
0563:     fpr, tpr, _ = roc_curve(y_test, prob_test)
0564:     precision, recall, _ = precision_recall_curve(y_test, prob_test)
0565:     auc = roc_auc_score(y_test, prob_test)
0566:     ap = average_precision_score(y_test, prob_test)
0567:
0568:     fig, axes = plt.subplots(1, 2, figsize=(11, 5))
0569:     axes[0].plot(fpr, tpr, color="#4C78A8", label=f"ROC-AUC = {auc:.3f}")
0570:     axes[0].plot([0, 1], [0, 1], "k--", alpha=0.5)
0571:     axes[0].set_xlabel("False positive rate")
0572:     axes[0].set_ylabel("True positive rate")
0573:     axes[0].set_title("ROC curve")
0574:     axes[0].legend()
0575:
0576:     axes[1].plot(recall, precision, color="#F58518", label=f"Average precision = {ap:.3f}")
0577:     axes[1].set_xlabel("Recall")
0578:     axes[1].set_ylabel("Precision")
0579:     axes[1].set_title("Precision–recall curve")
0580:     axes[1].legend()
0581:     fig.tight_layout()
0582:     fig.savefig(RESULTS_DIR / "06_lstm_roc_and_precision_recall_curves.png", dpi=180)
0583:     plt.close(fig)
0584:
0585:
0586: def plot_confusion_matrix(y_test: np.ndarray, pred_test: np.ndarray) -> None:
0587:     cm = confusion_matrix(y_test, pred_test)
0588:     fig, ax = plt.subplots(figsize=(5.5, 4.8))
0589:     sns.heatmap(
0590:         cm,
0591:         annot=True,
0592:         fmt="d",
0593:         cmap="Blues",
0594:         cbar=False,
0595:         xticklabels=["No event", "Event"],
0596:         yticklabels=["No event", "Event"],
0597:         ax=ax,
0598:     )
0599:     ax.set_xlabel("Predicted")
0600:     ax.set_ylabel("Actual")
0601:     ax.set_title("Test confusion matrix: 2023–2025")
0602:     fig.tight_layout()
0603:     fig.savefig(RESULTS_DIR / "07_lstm_confusion_matrix.png", dpi=180)
0604:     plt.close(fig)
0605:
0606:
0607: def plot_monthly_test_summary(predictions: pd.DataFrame) -> None:
0608:     monthly = predictions.groupby(predictions.index.month).agg(
0609:         actual_event_rate=("actual_event", "mean"), predicted_event_rate=("lstm_prediction", "mean")
0610:     )
0611:     fig, ax = plt.subplots(figsize=(9, 5))
0612:     months = np.arange(1, 13)
0613:     ax.bar(months - 0.18, monthly.reindex(months)["actual_event_rate"], width=0.36, label="Actual event rate",
       color="#F58518")
0614:     ax.bar(months + 0.18, monthly.reindex(months)["predicted_event_rate"], width=0.36, label="Predicted event
      rate", color="#4C78A8")
0615:     ax.set_xticks(months)
0616:     ax.set_xlabel("Month")
```

Appendix A: Complete Python Source Code

File: src/hydro_lstm_project.py

```
0617:     ax.set_ylabel("Event rate in test period")
0618:     ax.set_title("Does the model capture monsoon seasonality? Test period: 2023–2025")
0619:     ax.legend()
0620:     fig.tight_layout()
0621:     fig.savefig(RESULTS_DIR / "08_monthly_actual_vs_predicted_event_rate.png", dpi=180)
0622:     plt.close(fig)
0623:
0624:
0625: # ------------------------------
0626: # 5. Main workflow
0627: # ------------------------------
0628:
0629: def main() -> None:
0630:     raw_df = download_nasa_power_data()
0631:     df = add_hydrology_features(raw_df)
0632:     df.to_csv(DATA_DIR / "processed_powai_hydroclimate_2010_2025.csv")
0633:
0634:     bundle = make_lstm_dataset(df)
0635:     print("\nDataset summary")
0636:     print(f"Train: {bundle.X_train.shape}, event rate={bundle.y_train.mean():.3f}")
0637:     print(f"Validation: {bundle.X_val.shape}, event rate={bundle.y_val.mean():.3f}")
0638:     print(f"Test: {bundle.X_test.shape}, event rate={bundle.y_test.mean():.3f}")
0639:
0640:     model = NumpyLSTMClassifier(
0641:         input_size=bundle.X_train.shape[2], hidden_size=16, learning_rate=0.003, seed=42
0642:     )
0643:     history = model.fit(
0644:         bundle.X_train,
0645:         bundle.y_train,
0646:         bundle.X_val,
0647:         bundle.y_val,
0648:         epochs=20,
0649:         batch_size=64,
0650:     )
0651:     history.to_csv(RESULTS_DIR / "training_history.csv", index=False)
0652:
0653:     val_prob = model.predict_proba(bundle.X_val)
0654:     threshold, threshold_report = choose_threshold(bundle.y_val, val_prob)
0655:
0656:     test_prob = model.predict_proba(bundle.X_test)
0657:     test_pred = (test_prob >= threshold).astype(int)
0658:
0659:     predictions = pd.DataFrame(
0660:         {
0661:             "actual_event": bundle.y_test,
0662:             "lstm_probability": test_prob,
0663:             "lstm_prediction": test_pred,
0664:             "persistence_baseline": bundle.persistence_test,
0665:             "seasonal_monsoon_baseline": bundle.seasonal_test,
0666:         },
0667:         index=bundle.dates_test,
0668:     )
0669:     predictions.index.name = "forecast_date"
0670:     predictions.to_csv(RESULTS_DIR / "test_predictions_2023_2025.csv")
0671:
0672:     metrics = {
0673:         "project_location": "Powai / IIT Bombay, Mumbai, India",
0674:         "data_source": "NASA POWER daily point data",
0675:         "period": f"{START_DATE} to {END_DATE}",
0676:         "lookback_days": LOOKBACK_DAYS,
0677:         "target": f"Next-day runoff proxy >= {RUNOFF_EVENT_THRESHOLD_MM} mm",
0678:         "features": FEATURE_COLUMNS,
```

Appendix A: Complete Python Source Code

File: src/hydro_lstm_project.py

```
0679:         "train_period": "2010-2020",
0680:         "validation_period": "2021-2022",
0681:         "test_period": "2023-2025",
0682:         "validation_threshold_choice": threshold_report,
0683:         "selected_probability_threshold": threshold,
0684:         "lstm_test_metrics": binary_metrics(bundle.y_test, test_pred, test_prob),
0685:         "persistence_baseline_test_metrics": binary_metrics(bundle.y_test, bundle.persistence_test),
0686:         "seasonal_monsoon_baseline_test_metrics": binary_metrics(bundle.y_test, bundle.seasonal_test),
0687:     }
0688:     save_metrics(metrics)
0689:
0690:     # Plots
0691:     plot_monthly_climatology(df)
0692:     plot_annual_trend(df)
0693:     plot_monsoon_sample(df)
0694:     plot_training_loss(history)
0695:     plot_test_probability(predictions, threshold)
0696:     plot_roc_pr(bundle.y_test, test_prob)
0697:     plot_confusion_matrix(bundle.y_test, test_pred)
0698:     plot_monthly_test_summary(predictions)
0699:
0700:     print("\nSelected threshold from validation set:", round(threshold, 3))
0701:     print("LSTM test metrics:")
0702:     for key, value in metrics["lstm_test_metrics"].items():
0703:         print(f"  {key}: {value}")
0704:     print(f"\nSaved results in: {RESULTS_DIR}")
0705:
0706:
0707: if __name__ == "__main__":
0708:     main()
```

Appendix B: Python Package Requirements

```
numpy
pandas
matplotlib
seaborn
scikit-learn
requests
```